

Transformer Network (Part 3)

Introduction to Sequential Data Processing

Kyuhong Shim

Department of Intelligent Software
Sungkyunkwan University

2026. 4. 13.

- The standard Transformer already provides:
 - ▶ Global context modeling
 - ▶ Parallel training
 - ▶ Flexible encoder/decoder design
- Modern large-scale applications demand more:
 - ▶ Better modeling of **relative order** → relative positional encoding (RPE)
 - ▶ Better understanding of **longer contexts** → length extrapolation
 - ▶ More efficient **autoregressive inference** → key-value (KV) cache
 - ▶ Better **memory-quality tradeoff** during decoding → MHA, MQA, and GQA
 - ▶ More stable and effective **block designs** → RMSNorm, SwiGLU, QKNorm

- Absolute positional encoding (APE) adds a position-dependent but content-independent vector $\mathbf{p}_t$.

$$\tilde{\mathbf{x}}_t = \mathbf{x}_t + \mathbf{p}_t$$

- However, in many sequential problems, the **relative offset (distance)** is more important than the absolute position.
 - ▶ The model should distinguish the same content at different relative distances
 - ▶ The model should distinguish the same relative distance applied to different content.

- **Relative positional encoding (RPE)** is a function of the distance between query (t) and key (s) positions: $t - s$.
- **Additive RPE**: add a relative bias term to the attention score.
 - ▶ Shaw-style RPE
 - ▶ T5-style RPE
 - ▶ ALiBi RPE
- **Multiplicative RPE**: modify the query/key vectors directly.
 - ▶ Rotary Positional Embedding (RoPE)

- **Shaw-style** RPE adds a learnable embedding to each relative offset:

$\mathbf{r}_{t-s}$ (trainable embedding)

- The attention score is modified as:

$$s_{t,s} = \frac{\mathbf{q}_t^\top \mathbf{k}_s + \mathbf{q}_t^\top \mathbf{r}_{t-s}}{\sqrt{d}}$$

- ▶ $\mathbf{q}_t^\top \mathbf{k}_s$ measures content similarity.
- ▶ $\mathbf{q}_t^\top \mathbf{r}_{t-s}$ injects relative position information.
- ▶ Here, relative position information interacts with the query.
- ▶ In practice, relative distances are usually clipped to a fixed range because the embedding table is finite.

- Shaw-style RPE is difficult to implement efficiently because the positional term depends on the query vector.
 - It is possible to parallelize, but the implementation is inefficient.
- T5-style RPE** disentangles the positional bias from the query.
- The attention score is modified as:

$$s_{t,s}^{(h)} = \frac{\mathbf{q}_t^{(h)\top} \mathbf{k}_s^{(h)}}{\sqrt{d}} + r_{t-s}^{(h)}$$

r_0			
r_{-1}	r_0		
r_{-2}	r_{-1}	r_0	
r_{-3}	r_{-2}	r_{-1}	r_0

- $r^{(h)}$ is a scalar bias for head h .
- The RPE bias can now be added directly to the mask $\mathbf{M}$.

- **ALiBi** adds a simple distance-dependent bias to the score:

$$s_{t,s}^{(h)} = \frac{\mathbf{q}_t^{(h)\top} \mathbf{k}_s^{(h)}}{\sqrt{d}} - k_h(t - s)$$

- ▶ $k_h > 0$ is a slope assigned to the h -th attention head.
- ▶ For example, $\mathbf{k} = \{1/2, 1/4, \dots, 1/256\}$.
- ▶ Large relative distance leads to a larger *penalty*.
- ▶ No learnable parameters are required.

$k_h \times$

0			
-1	0		
-2	-1	0	
-3	-2	-1	0

- Shaw, T5, and ALiBi all modify attention scores by adding relative-position information.

- **RoPE rotates** the query and key vectors according to their positions.
 - ▶ RoPE is not an additive RPE.
 - ▶ RoPE does not include trainable parameters.
- For each position t or s , apply a position-dependent transformation:

$$\tilde{\mathbf{q}}_t = \mathbf{R}_t \mathbf{q}_t, \quad \tilde{\mathbf{k}}_s = \mathbf{R}_s \mathbf{k}_s$$

- The attention score computation becomes:

$$e_{t,s} = \frac{\tilde{\mathbf{q}}_t^\top \tilde{\mathbf{k}}_s}{\sqrt{d}} = \frac{\mathbf{q}_t^\top \mathbf{R}_t^\top \mathbf{R}_s \mathbf{k}_s}{\sqrt{d}}$$

- RoPE uses a special $\mathbf{R}$ that $\mathbf{R}_t^\top \mathbf{R}_s$ depends only on the relative offset $s - t$.

- RoPE applies a 2D rotation to each pair of coordinates:

$$\begin{bmatrix} \tilde{x}_{2i} \\ \tilde{x}_{2i+1} \end{bmatrix} = \begin{bmatrix} \cos \theta_{t,i} & -\sin \theta_{t,i} \\ \sin \theta_{t,i} & \cos \theta_{t,i} \end{bmatrix} \begin{bmatrix} x_{2i} \\ x_{2i+1} \end{bmatrix}.$$

- The angle $\theta_{t,i}$ uses the same frequency schedule as sinusoidal PE.

$$\mathbf{R}_t = \begin{bmatrix} \cos \theta_{t,0} & -\sin \theta_{t,0} & 0 & 0 & \cdots & \cdots & 0 & 0 \\ \sin \theta_{t,0} & \cos \theta_{t,0} & 0 & 0 & \cdots & \cdots & 0 & 0 \\ 0 & 0 & \cos \theta_{t,1} & -\sin \theta_{t,1} & \cdots & \cdots & 0 & 0 \\ 0 & 0 & \sin \theta_{t,1} & \cos \theta_{t,1} & \cdots & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & & & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cdots & \cos \theta_{t,d/2-1} & -\sin \theta_{t,d/2-1} \\ 0 & 0 & 0 & 0 & \cdots & \cdots & \sin \theta_{t,d/2-1} & \cos \theta_{t,d/2-1} \end{bmatrix} \in \mathbb{R}^{d \times d}$$

- The core property of the rotation matrix is:

$$\mathbf{R}_t^\top \mathbf{R}_s = \mathbf{R}_{s-t}$$

- Therefore,

$$\tilde{\mathbf{q}}_t^\top \tilde{\mathbf{k}}_s = (\mathbf{R}_t \mathbf{q}_t)^\top (\mathbf{R}_s \mathbf{k}_s) = \mathbf{q}_t^\top \mathbf{R}_{s-t} \mathbf{k}_s$$

- ▶ The score depends on the relative offset $s - t$, not just on the absolute positions.
- ▶ Lower-index coordinate pairs rotate faster, while higher-index pairs rotate more slowly.
- Because materializing and multiplying $\mathbf{R}$ is too expensive, we directly compute the rotated $\tilde{\mathbf{q}}_t$ and $\tilde{\mathbf{k}}_s$ in practice.

- Can the model generalize to sequences longer than those seen during training?
- **Length extrapolation** is a technique to address this problem.
 - ▶ APE often struggles because unseen positions require unseen embeddings.
 - ▶ RPE is more robust because they depend more directly on relative offsets.
- Post-training length extrapolation techniques for RoPE:
 - ▶ **Position interpolation**: replace $t \rightarrow t/c$ where $c > 1$, leading to slower rotation.
 - ▶ **Base conversion**: increase the base of θ so that high-frequency rotation is weakened at long-context lengths. This often requires additional fine-tuning.

$$\theta = \frac{t}{10000^{2i/d}} \longrightarrow \frac{t}{1000000^{2i/d}}$$

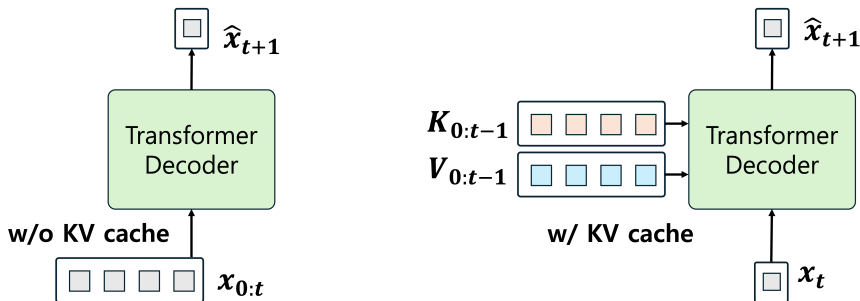
- During AR generation, at step t we predict the next token from the previous context:

$$\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_t \longrightarrow \mathbf{x}_{t+1}$$

- In causal self-attention, past keys and values do not change after they are computed.
- Without caching, each new step recomputes attention over the entire prefix.
 - ▶ Computation is repeated across steps.
 - ▶ Latency accumulates over long outputs.

- Instead of recomputing past $\mathbf{K}$ and $\mathbf{V}$ at every step, we store them in a **KV cache**.
 - ▶ Compute only the new $\mathbf{q}_t, \mathbf{k}_t, \mathbf{v}_t$.
 - ▶ Append $\mathbf{k}_t, \mathbf{v}_t$ to the KV cache.
 - ▶ Attend using the cached past keys/values.

$$\begin{aligned}
 \mathbf{s}_t &= \frac{\mathbf{q}_t^\top \overbrace{[\mathbf{k}_0, \mathbf{k}_1, \dots, \mathbf{k}_t]}^{\text{Key cache after update}}}{\sqrt{d}} \\
 \mathbf{y}_t &= \sum_{k=0}^t \alpha_{t,k} \mathbf{v}_k = \overbrace{[\mathbf{v}_0, \mathbf{v}_1, \dots, \mathbf{v}_t]}^{\text{Value cache after update}} \alpha_t^\top
 \end{aligned}$$



- Each SA (not CA) layer has its own KV cache.
 - ▶ The Key and Value cache are 4D Tensors, whose shapes are $(L, H, t - 1, d_h)$.
 - ▶ After processing step t , the cache will be updated to (L, H, t, d_h) .

- Suppose the cached tensors up to step $t - 1$ are:

$$\mathbf{K}_{0:t-1} = [\mathbf{k}_0, \mathbf{k}_1, \dots, \mathbf{k}_{t-1}], \quad \mathbf{V}_{0:t-1} = [\mathbf{v}_0, \mathbf{v}_1, \dots, \mathbf{v}_{t-1}]$$

- At the new step t , compute $\mathbf{q}_t, \mathbf{k}_t, \mathbf{v}_t$ and update the cache:

$$\mathbf{K}_{0:t} = \text{Concat}(\mathbf{K}_{0:t-1}, \mathbf{k}_t), \quad \mathbf{V}_{0:t} = \text{Concat}(\mathbf{V}_{0:t-1}, \mathbf{v}_t)$$

- The attention computation becomes:

$$\mathbf{s}_t = \frac{\mathbf{q}_t^\top \mathbf{K}_{0:t}}{\sqrt{d}} \in \mathbb{R}^{1 \times (t+1)}, \quad \boldsymbol{\alpha}_t = \text{softmax}(\mathbf{s}_t) \in \mathbb{R}^{1 \times (t+1)}, \quad \mathbf{y}_t = \mathbf{V}_{0:t} \boldsymbol{\alpha}_t^\top \in \mathbb{R}^{d \times 1}$$

$$\mathbf{y}_t = \text{Attention}(\mathbf{q}_t, \mathbf{K}_{0:t}, \mathbf{V}_{0:t}).$$

- **Benefit**

- ▶ Reduces repeated computation during decoding
- ▶ Lowers per-step latency (i.e., computation bottleneck)

- **Cost**

- ▶ KV cache memory grows with sequence length (i.e., memory bottleneck)
- ▶ KV cache load/save/update logic must be added to the inference code, which increases implementation complexity.

Example

Consider a 16-layer decoder-only Transformer with $d = 512$, $H = 8$, and $d_h = 64$. Compute the total KV cache size after processing $t = 0, 1, 2, \dots, 99$.

- In standard multi-head attention, each head has its own Q, K, V projections:

$$Q^{(h)} = \mathbf{X}W_Q^{(h)}, \quad K^{(h)} = \mathbf{X}W_K^{(h)}, \quad V^{(h)} = \mathbf{X}W_V^{(h)}$$

- ▶ Each head stores its own KV cache.
- ▶ This design maximizes the expressiveness (i.e., each head learns different relationships), but the total KV cache memory linearly grows as H increases.

- **Multi-Query Attention (MQA)** keeps multiple query heads, but shares **one key and one value** across all heads.
- Queries remain head-specific:

$$\mathbf{Q}^{(h)} = \mathbf{XW}_Q^{(h)} \in \mathbb{R}^{T \times d_h}, \quad \mathbf{Q} = \mathbf{XW}_Q \in \mathbb{R}^{T \times (H \times d_h)} \quad (\text{concatenated form})$$

- But keys and values are shared:

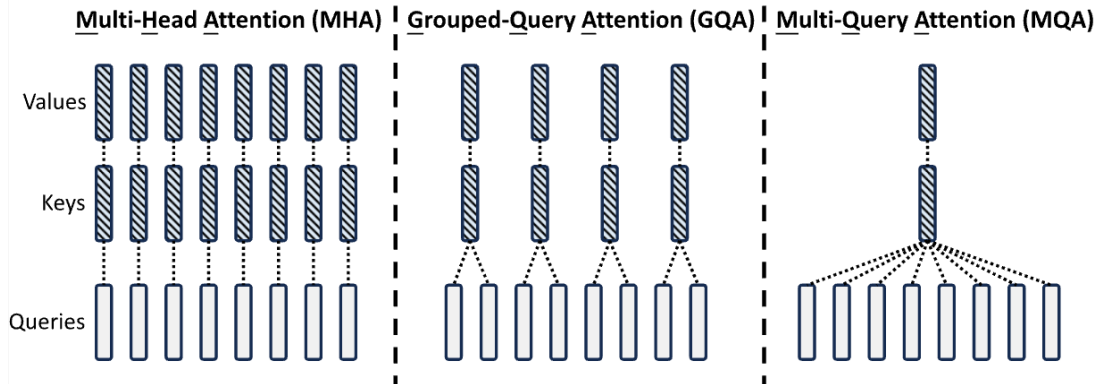
$$\mathbf{K} = \mathbf{XW}_K \in \mathbb{R}^{T \times d_h}, \quad \mathbf{V} = \mathbf{XW}_V \in \mathbb{R}^{T \times d_h}$$

- Characteristics:
 - ▶ Much smaller KV cache and smaller parameter size
 - ▶ Faster decoding
 - ▶ Less expressive than full MHA (i.e., lower performance)

- **Grouped-Query Attention (GQA)** strikes a balance between MHA and MQA.
- Several query heads share **one key/value group**.
 - ▶ Let there be H query heads and G key-value groups, where $G < H$.
 - ▶ Each group has one KV that serves multiple Q in the group.
- Characteristics:
 - ▶ Smaller KV cache than MHA
 - ▶ More expressive than MQA
 - ▶ GQA is the standard for modern large Transformer decoder models

Example

A GQA block has 64 query heads and 16 groups. In terms of KV cache, compute the memory savings of this block compared to full MHA.



- Many modern Transformers replace LayerNorm with **RMSNorm**.
- RMSNorm normalizes by the root-mean-square magnitude:

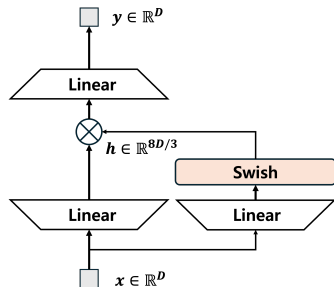
$$\text{RMSNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x}}{\sqrt{\frac{1}{d} \|\mathbf{x}\|^2 + \varepsilon}}.$$

- Compared with LayerNorm:
 - ▶ No mean subtraction and β offset
 - ▶ Only scale normalization and γ scaling
 - ▶ Simpler and often effective in practice
- RMSNorm does not explicitly remove the mean, while still controlling feature scale.
 - ▶ Similar to RMSNorm, many modern Transformers omit linear-layer biases in practice.

- Modern Transformers often replace the standard FFN with variants such as **SwiGLU**.

$$\text{SwiGLU}(\mathbf{x}) = \mathbf{W}_{\text{down}} \left((\mathbf{W}_{\text{up}} \mathbf{x}) \odot \text{swish}(\mathbf{W}_{\text{gate}} \mathbf{x}) \right)$$

- Intuition:
 - Up-projection branch transforms features
 - Gate projection adjusts each features



- **QKNorm** normalizes queries and keys before computing attention scores.
- For each token and head, vectors are ℓ_2 -normalized:

$$\hat{\mathbf{q}}_t = \frac{\mathbf{q}_t}{\|\mathbf{q}_t\|_2}, \quad \hat{\mathbf{k}}_s = \frac{\mathbf{k}_s}{\|\mathbf{k}_s\|_2}.$$

- Then the attention score becomes:

$$s_{t,s}^{(h)} = \tau_h \hat{\mathbf{q}}_t^{(h)\top} \hat{\mathbf{k}}_s^{(h)} \quad (\tau_h : \text{learnable scale (a.k.a. temperature)})$$

- Intuition:
 - ▶ The dot product becomes close to cosine similarity.
 - ▶ The score depends less on vector magnitude and more on angular alignment.
 - ▶ QKNorm is known to stabilize attention scores and training for large Transformers.

- A modern Transformer block often looks like:

$$\mathbf{H}' = \mathbf{X} + \text{Attention}(\text{RMSNorm}(\mathbf{X}))$$

$$\mathbf{Y} = \mathbf{H}' + \text{SwiGLU}(\text{RMSNorm}(\mathbf{H}'))$$

- The attention part may also include:
 - ▶ **RoPE** for positional encoding
 - ▶ **GQA** for smaller KV cache
 - ▶ **QKNorm** for training stability
 - ▶ **KV cache** during inference
- These modifications are designed for:
 - ▶ Relative positional information and longer context support
 - ▶ Faster inference and better memory/parameter efficiency
 - ▶ Strong empirical performance